

规划

祝恩
计算机学院
国防科技大学

学习内容

1. 用PDDL将规划问题**定义**为一个搜索问题
2. 前向**搜索**、后向**搜索**，及搜索的**启发式**
3. 从**规划图**获得启发式及提取规划

学习目标

1. 能够用PDDL**描述**一个规划问题
2. 能够**对比**规划Agent与基于搜索的问题求解Agent
3. 能够**写出**前向搜索的状态空间转移公式和后向搜索的相关状态计算公式
4. 能够**解释**规划中的几种启发式
5. **解释**从规划图获得启发式和提取规划

回顾

ch03基于搜索的问题求解Agent处理状态的原子表示
ch07的逻辑Agent依赖于基元命题推理(不含变量)

Ch10处理状态的要素化表示和含变量的推理

Ch10考虑完全可观察的、确定性的、静态的、单Agent环境

1 用PDDL将规划问题 定义为一个搜索问题



航空货物运输

- 航空货物运输问题：货物C1和飞机P1在SFO机场，货物C2和飞机P2在JFK机场。将C1运到JFK，将C2运到SFO。
- 动作Load, Unload, Fly
- solution: [$Load(C_1, P_1, SFO)$, $Fly(P_1, SFO, JFK)$, $Unload(C_1, P_1, JFK)$, $Load(C_2, P_2, JFK)$, $Fly(P_2, JFK, SFO)$, $Unload(C_2, P_2, SFO)$]



A.规划问题的形式化

PDDL

- 将规划问题定义为一个搜索问题
- 用PDDL描述：
- 状态、动作、转移模型、初始状态、目标



(1) 状态的表示

状态的表示：流的合取

- **流**是基元的（无变量的）、无函数的原子。
 - *Poor, Unknown, At(P1,SFO), At(P2,JFK)*
 - 不是流：
 $At(x,y)$ 、 $\neg Poor$ 、 $At(Father(Fred), Sydney)$
- **状态**表示为**流**（fluent）的合取。（或者表示为流的集合）
 - 例如，穷且无知的Agent的状态：
 $Poor \wedge Unknown$
 例如， $At(P1,SFO) \wedge At(P2,JFK)$
 - 如何表示状态 $\neg Poor \wedge Unknown$?



使用数据库语义

- 使用数据库语义。



(2) 动作的表示

动作模式

□ 动作可用**动作模式**来描述。

■ 例如直升机从一个地点飞行到另一个地点的动作模式：

$Action(Fly(p, from, to),$

PRECOND: $At(p, from) \wedge Plane(p) \wedge Airport(from)$
 $\wedge Airport(to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

□ PDDL根据什么发生了变化来描述一个动作的效果；不提
 及保持不变的东西。动作的前提和效果都是文字（原子
 语句或原子语句的否定）的合取。前提定义了动作能被
 执行的状态，效果定义了执行这个动作的结果。

基元动作

- 通过为所有的变量代入值而得到**基元动作**：

$Action(Fly(P_1, SFO, JFK),$

PRECOND: $At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO)$
 $\wedge Airport(JFK)$

EFFECT: $\neg At(P_1, SFO) \wedge At(P_1, JFK)$

- 动作模式隐式描述了**ACTIONS(s)**和**RESULT(s, a)**



(3) ACTIONS(S)

ACTIONS(s)

- ACTIONS(s): 一个动作 a 能在状态 s 被执行, 当且仅当 s 蕴涵了 a 的前提。用形式符号就是:
 - $(a \in \text{ACTIONS}(s)) \Leftrightarrow s \models \text{PRECOND}(a),$

蕴涵的集合语义

- 蕴涵也可用集合语义来表达： $s \models q$ 当且仅当 q 中的正文字都在 s 中，且 q 中的负文字都不在 s 中。

适用的

- 如果状态 s 满足 a 的前提, $a \in \text{ACTIONS}(s)$, 即 $s \models \text{PRECOND}(a)$, 我们称在状态 s 动作 a 是**适用的** (applicable)。



(4) RESULT(S,A)

RESULT(s, a)

- RESULT(s, a): 在状态 s 执行动作 a 的结果定义为状态 s' , :
- $s' = \text{RESULT}(s, a) = (s - \text{_____}) \cup \text{_____}$

Action(Fly(P_1 , SFO, JFK),
 PRECOND: $\text{At}(P_1, \text{SFO}) \wedge \text{Plane}(P_1) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$
 EFFECT: $\neg \text{At}(P_1, \text{SFO}) \wedge \text{At}(P_1, \text{JFK})$)

ADD() = ? DEL () = ?



(5) 初始状态和目标

初始状态和目标

- **初始状态**是基元原子的合取。
- 对于所有状态，使用**封闭世界假设**：任何没有提到的原子都是假的。
- **目标**就像前件：是可以含有变量的文字（正文字或负文字）的合取，像 $At(p, SFO) \wedge Plane(p)$ 。
。 **变量是存在量化的**

目标测试

- 当我们能找到一个动作序列，使得在蕴涵了目标的状态 s 结束，问题就得到了解决。例如，状态 $Plane(Plane_1) \wedge At(Plane_1, SFO)$ 蕴涵了目标 $At(p, SFO) \wedge Plane(p)$ 。

目标测试：测试_____

规划定义为一个搜索问题

- 一组动作模式就定义了一个规划域
- 加上一个初始状态和一个目标就定义了这个规划域中一个特定的问题。

实例

Init ($\text{At}(C_1, \text{SFO}) \wedge \text{At}(C_2, \text{JFK}) \wedge \text{At}(P_1, \text{SFO}) \wedge \text{At}(P_2, \text{JFK})$
 $\wedge \text{Cargo}(C_1) \wedge \text{Cargo}(C_2) \wedge \text{Plane}(P_1) \wedge \text{Plane}(P_2)$
 $\wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO}))$

Goal ($\text{At}(C_1, \text{JFK}) \wedge \text{At}(C_2, \text{SFO})$)

Action(Load (c, p, a),

PRECOND: $\text{At}(c, a) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\neg \text{At}(c, a) \wedge \text{In}(c, p)$)

Action(Unload(c, p, a),

PRECOND: $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\text{At}(c, a) \wedge \neg \text{In}(c, p)$)

Action(Fly(p, from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

solution: [*Load*(C_1, P_1, SFO), *Fly*($P_1, \text{SFO}, \text{JFK}$),
Unload(C_1, P_1, JFK), *Load*(C_2, P_2, JFK), *Fly*($P_2, \text{JFK}, \text{SFO}$),
Unload(C_2, P_2, SFO)]

如何求解用PDDL描述的一个规划问题

2 前向搜索、后向搜索， 搜索的启发式





A.前向状态空间搜索

RESULT(s, a)

- 规划问题的描述定义了一个搜索问题，启发式搜索算法（第3章）或局部搜索算法（第4章）可求解规划问题。
- 前向搜索从初始状态开始搜索状态空间，寻找一个目标。
- $s' = \text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$

容易探索到无关动作

- 从规划搜索的初期（约1961年）直到约1998年，人们认为前向状态空间搜索太低效而不能实际应用。
- 首先，前向搜索**容易探索到无关动作**。
 - 考虑从在线书店购买《AI: A Modern Approach》这本书。有一个动作模式 $Buy(isbn)$ ，效果是 $Own(isbn)$ 。ISBN是10位数，因此这个动作模式表示100亿个基元动作。**无启发的**前向搜索将需要枚举这100亿个基元动作，以找到通向目标的动作。

状态空间大

- 其次，规划问题经常有**大的状态空间**，从而需要精确的**启发式**。
 - 尽管规划的许多现应用依赖于特定领域的启发知识，还是能自动导出非常强的独立于领域的启发知识；这使得前向搜索具有可行性。



B. 后向相关状态搜索

只考虑相关的动作

- 从目标开始，向后应用动作，直到找到达到初始状态的步骤序列。
- 只考虑与当前目标状态**相关的动作**（**导致当前目标状态的规划中可以作为最后一个步骤的那些动作**）。每一步考虑一组相关状态(就像信念状态(ch04))，而不是单个状态。

g经过a如何回退到g'

- 假设g为D, 动作a的前提是 $A \wedge B \wedge C$, 效果为 $D \wedge \neg C$
- $g' = (g - \text{ADD}(a)) \cup \text{DEL}(a) = C$, 显然g'不满足a的前提。
- $g' = (g - \text{ADD}(a)) \cup \text{Precond}(a) = A \wedge B \wedge C$, 显然g'满足a的前提。



在前向搜索中,

$$s' = \text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$$



哪些动作是相关的？

- 哪些动作是相关的？
- 导致当前目标状态的规划中可以作为最后一个步骤的那些动作
- 如果目标是 $A \wedge B \wedge C$,
 一个动作的效果是 $A \wedge B \wedge \neg C$, 那么它是相关的吗?
 一个动作的效果是 D , 那么它是相关的吗?



后向搜索实例

- 给定目标 $At(C_2, SFO)$, $Unload(c, p, a)$ 的几个实例是相关的：可以选择任何特定的飞机来卸载，或者通过使用动作 $Unload(C_2, p', SFO)$ 而可以不指定飞机。
- 通过总是使用使目标和效果合一的最一般合一元代换到（标准化分离的）动作模式中得到的动作，我们可以减小分支因子而不会漏掉任何解。

Action($Unload(c, p, a)$,
 PRECOND: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$
 EFFECT: $At(c, a) \wedge \neg In(c, p)$)



C. 规划中的启发式

(1)忽略所有前提

- 去掉所有前提：
- 忽略前提启发式（增加边/松弛前提）
丢弃动作中的所有前提。在每一个状态，每一个动作都变得适用，而且任何单个目标流能够一步获得（如果存在一个适用动作——否则，问题是不可能解的）

这几乎蕴含了：求解这个松弛问题所需的步数等于未被满足的目标数——几乎，而不是完全。

(1) 一些动作可能实现多个目标，

(2) 一些动作可能抵消另外一些动作的效果。

对于许多问题，一个准确的启发式是通过考虑 (1) 而忽略 (2) 而获得的。

忽略所有前提以及目标以外的所有效果

- 去掉所有前提和目标文字以外的所有效果：
- 因为一个动作可能抵消另一个动作的效果。因此对于许多问题，可以去掉所有前提以及目标文字以外的所有效果。然后，统计需要的最小动作数，使这些动作的效果的并集满足目标。 - --这个松弛问题的最优解问题本质上是一个**集合覆盖问题**，是NP难的，存在贪婪算法，但贪婪算法不能保证可采纳性。

忽略部分前提

- **选择性地忽略动作的前提**是可能的。
考虑8码问题。将它编制为一个使用单个动作模式 *Slide* 的规划问题：
- $Action(Slide(t, s_1, s_2),$
 $PRECOND: On(t, s_1) \wedge Tile(t) \wedge Blank(s_2) \wedge Adjacent(s_1, s_2)$
 $EFFECT: On(t, s_2) \wedge Blank(s_1) \wedge \neg On(t, s_1) \wedge \neg Blank(s_2))$
- 去掉前提_____，那么一个动作中任何数字块能够移到任何方格，从而得到位置错误的数字块数作为启发式

忽略部分前提

- 选择性地忽略动作的前提是可能的。
考虑8码问题。将它编制为一个使用单个动作模式 *Slide* 的规划问题：
- $Action(Slide(t, s_1, s_2),$
 PRECOND: $On(t, s_1) \wedge Tile(t) \wedge Blank(s_2) \wedge Adjacent(s_1, s_2)$
 EFFECT: $On(t, s_2) \wedge Blank(s_1) \wedge \neg On(t, s_1) \wedge \neg Blank(s_2))$
- 去掉前提 $Blank(s_2) \wedge Adjacent(s_1, s_2)$, 那么一个动作中任何数字块能够移到任何方格, 从而得到位置错误的数字块数作为启发式

(2)忽略删除列表

- 考虑到一个动作可能抵消另一个动作的效果，可考虑**忽略删除列表**（增加边/松弛效果）启发式，这使得没有动作会抵消另一个动作的效果，从而使得解短于原始空间的解。
- 这个松弛问题的最优解问题是NP难的，但用**爬山法**可在多项式时间找到近似解。

(3) 忽略一些流

- 对动作进行松弛不会减少状态数量，求解代价仍很高。
- 通过形成**状态抽象** (state abstraction) 而减少状态数的松弛方法：
- 状态抽象的最简单方式是**忽略一些流**
- 抽象状态空间中的一个解将短于原始空间中的一个解（因而是可采纳的启发式）

(4)目标分解

- **目标分解**：将一个问题分成多个部分，独立地解决每个部分，再组合各部分。
- 假设目标是一组流 G ，我们将其划分为不相交的子集 $G_1, \dots, G_n$ 。然后我们找到各子目标的解规划 $P_1, \dots, P_n$ 。

max和plus

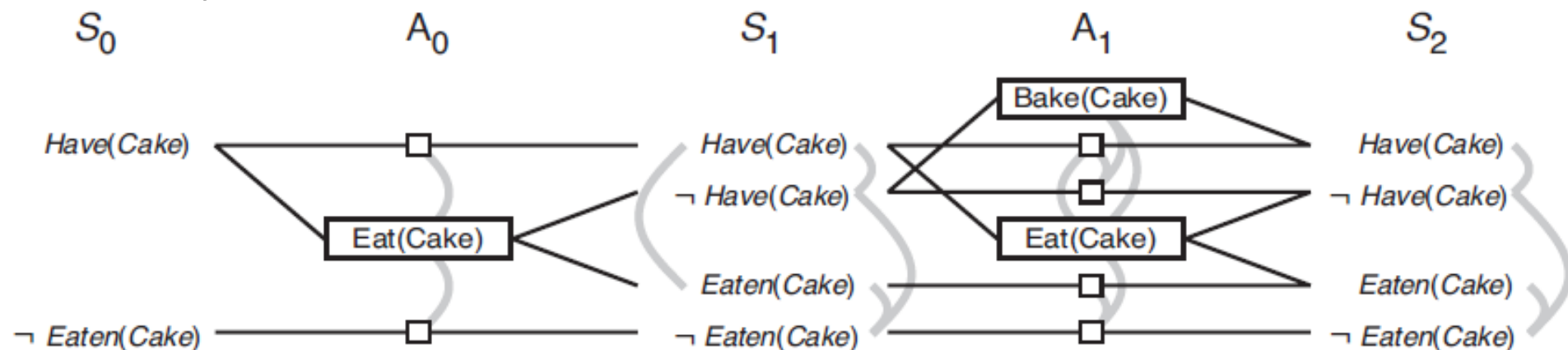
- **最大估计值** $\max_i \text{COST}(P_i)$ 是可采纳的，而且有时是精确正确的： P_1 可能偶然实现所有 G_i 。但大多数情况下，实际估计值过低。
- 最好的情况是当我们确定 G_i 和 G_j 相互独立的时候：如果 P_i 的效果不改变 P_j 的所有前提和目标，那么估计值 $\text{COST}(P_i) + \text{COST}(P_j)$ 是可采纳的，而且比最大估计值更准确。

3 从规划图 获得启发式及提取规划



何谓规划图?

- 一个**规划图**组织成**有层次的**有向图：首先是初始状态层 S_0 ，由在 S_0 成立的每个流结点组成；然后是层次 A_0 ，由在 S_0 适用的每个基元动作结点组成；然后是交叠的层次 S_i ，后面紧跟层次 A_i ；直到达到终止条件（是什么？）。
- 规划图只能用于命题规划问题——**没有变量的规划问题**



“蛋糕问题”的规划图

□ 蛋糕问题

Init (Have(Cake))

Goal (Have(Cake) $\wedge$ Eaten(Cake))

Action(Eat(Cake))

PRECOND: Have(Cake)

EFFECT: $\neg$ Have(Cake) $\wedge$ Eaten(Cake))

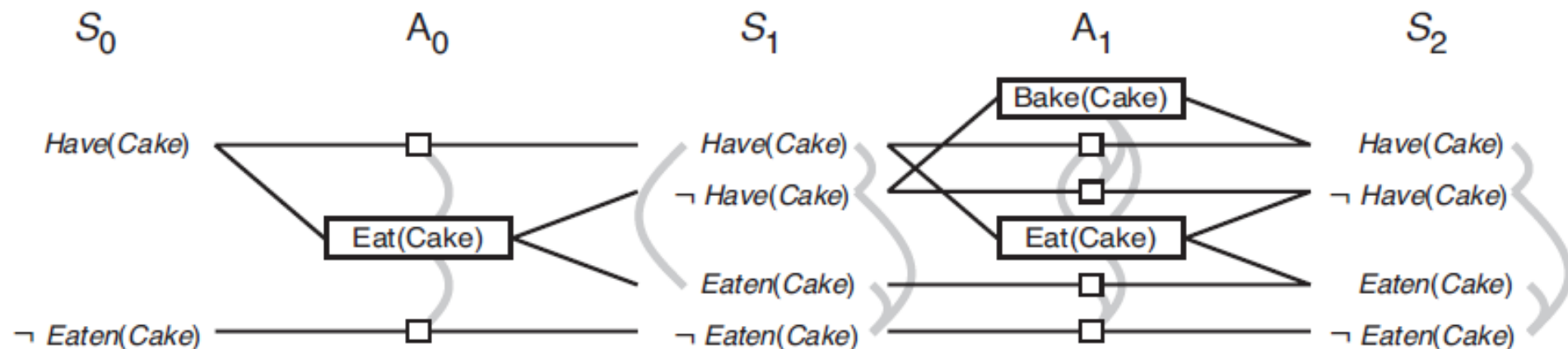
Action(Bake(Cake))

PRECOND: $\neg$ Have(Cake)

EFFECT: Have(Cake))

“蛋糕问题”的规划图

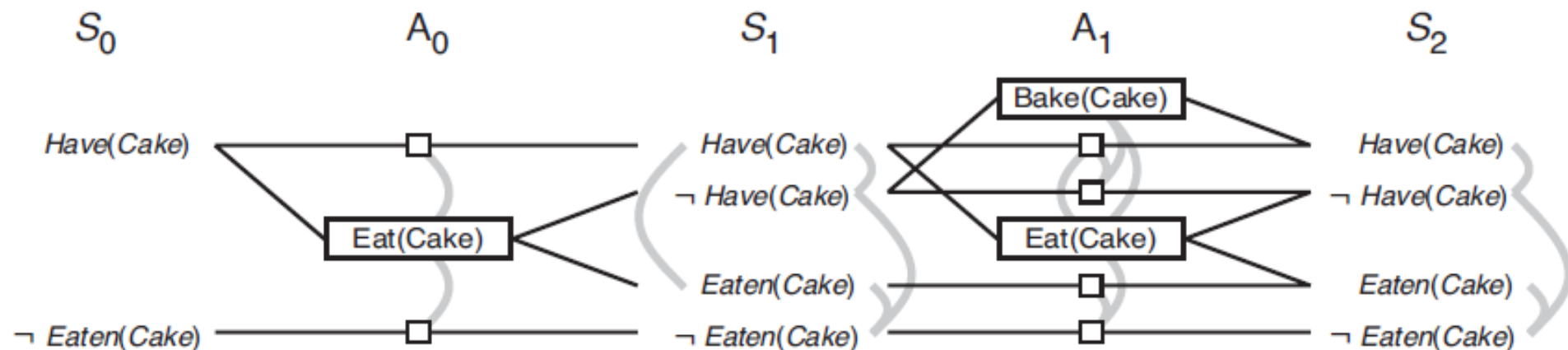
- 矩形代表动作（小方格代表持续动作），直线代表前提和效果。灰色曲线表示互斥连接。并不是所有互斥连接都画出来了，那样会显得太零乱。一般，如果在 S_i 两个文字是互斥的，那么在 A_i 这些文字的持续动作将是互斥的，我们不需要画出互斥连接



达到稳定

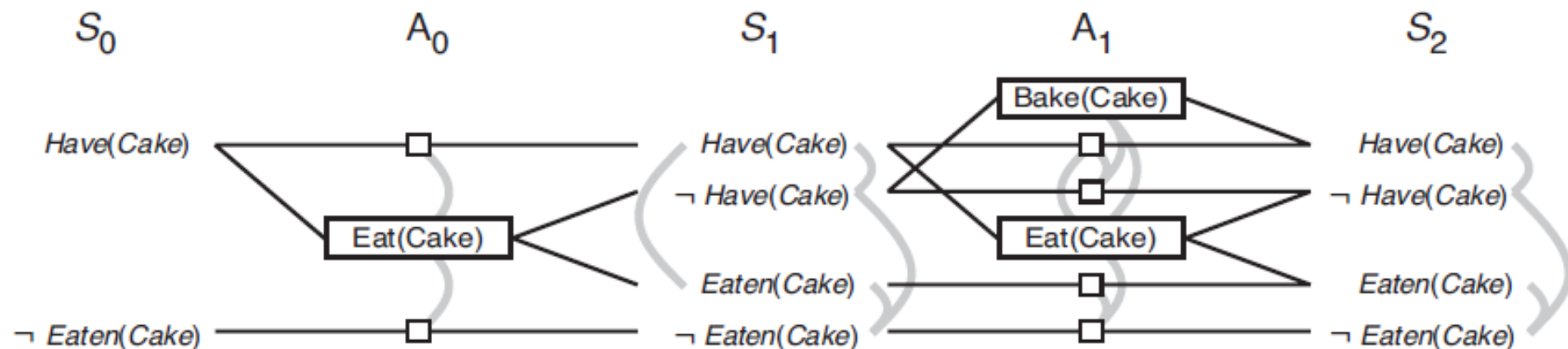
- 在状态层 S 和动作层 A_i 之间交叠，直到出现连续两层是一模一样的，此时，就说规划图达到了稳定

在 S_2 达到了稳定



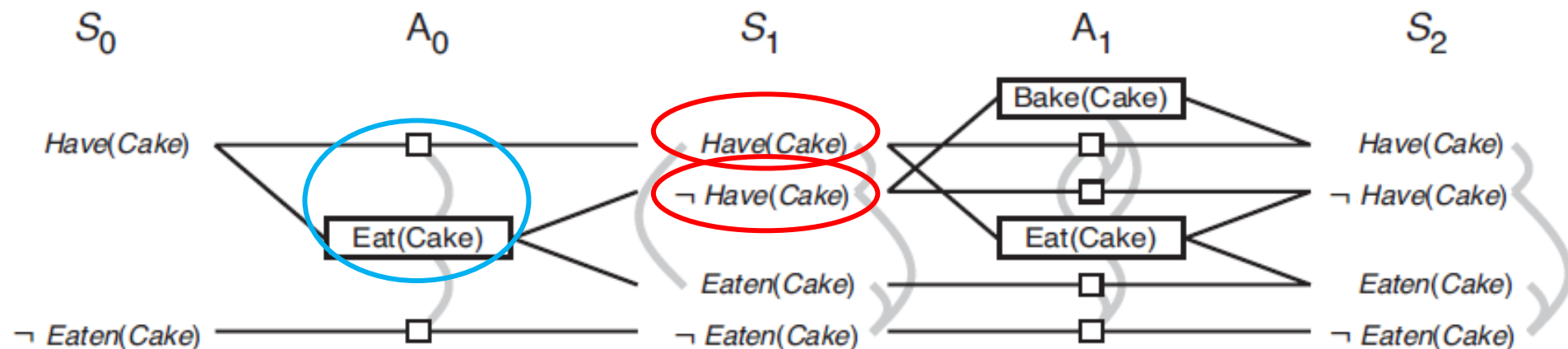
达到稳定

- 最后得到一个结构，其中每个 A_i 层含有 S_i 中适用的所有动作，以及两个动作不能在同一层都被执行的约束。每个 S_i 层包含 A_{i-1} 中动作的可能选择可能导致的所有文字，以及哪些文字对不能成对出现的约束。



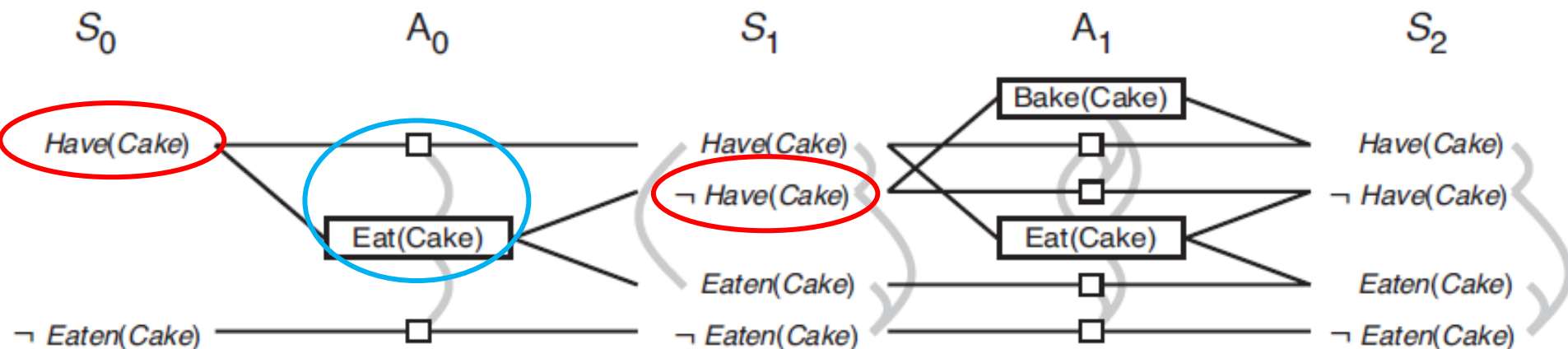
什么情况下两个动作是互斥的？

- 在给定层次的两个动作间的互斥关系成立，如果下列三个条件之一成立：
 1. **不一致效果**：一个动作的效果否定另一个动作的效果。例如， $Eat(Cake)$ 和 $Have(Cake)$ 的持续动作具有不一致效果。



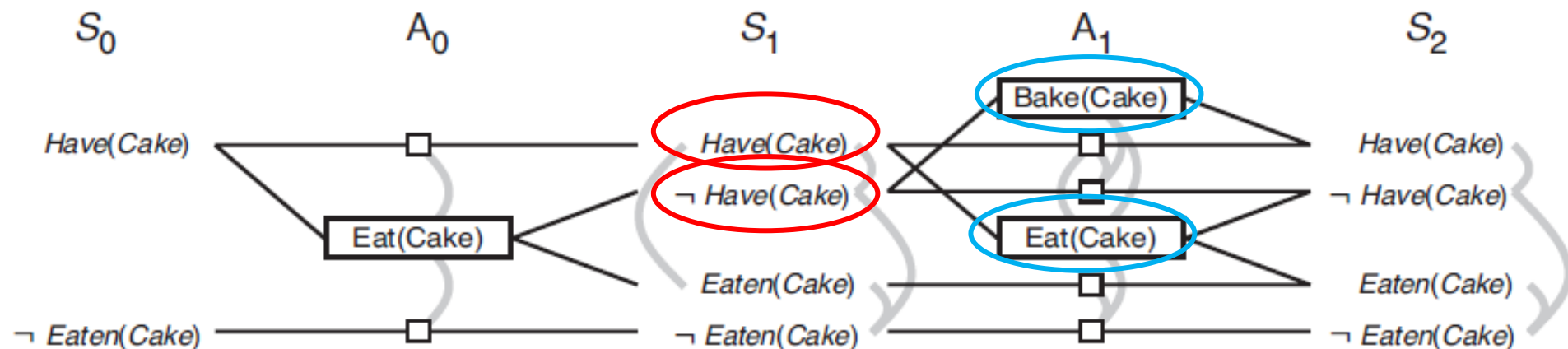
什么情况下两个动作是互斥的？

- 在给定层次的两个动作间的互斥关系成立，如果下列三个条件之一成立：
 2. **冲突**：一个动作的效果否定另一个动作的前提。例如， $Eat(Cake)$ 与 $Have(Cake)$ 的持续动作冲突，因为 $Eat(Cake)$ 否定了它的前提。



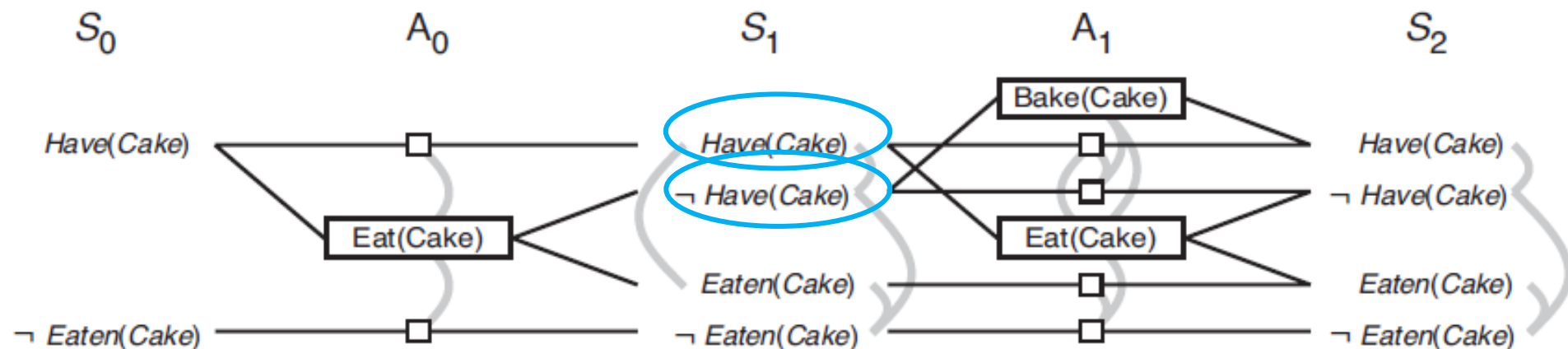
什么情况下两个动作是互斥的？

- 在给定层次的两个动作间的互斥关系成立，如果下列三个条件之一成立：
- 3. **竞争需要**：一个动作的前提之一与另一个动作的一个前提互斥。例如， $Bake(Cake)$ 和 $Eat(Cake)$ 是互斥的，因为它们对前提 $Have(Cake)$ 的值是竞争的。



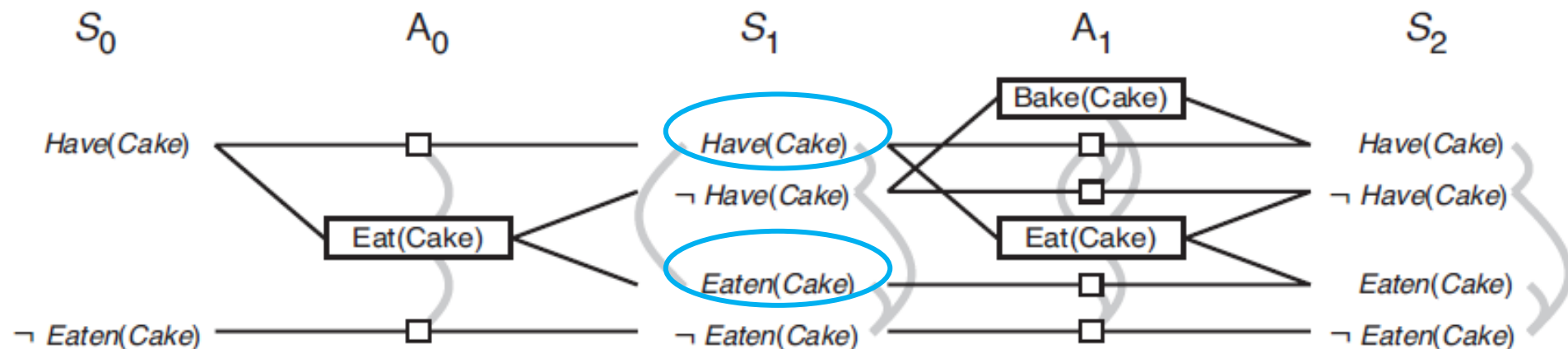
什么情况下两个文字是互斥的？

- 同一层的两个文字之间的互斥关系成立：
 1. 如果一个文字是另一个文字的负



什么情况下两个文字是互斥的？

- 同一层的两个文字之间的互斥关系成立：
- 2. 或者得到这两个文字的每对可能的动作是互斥的。这个条件称为非一致性支持 (inconsistent support)。例如， $Have(Cake)$ 和 $Eaten(Cake)$ 在 S_1 处是互斥的，因为获得 $Have(Cake)$ 的唯一途径——持续动作——与获得 $Eaten(Cake)$ 的唯一途径—— $Eat(Cake)$ ——是互斥的。





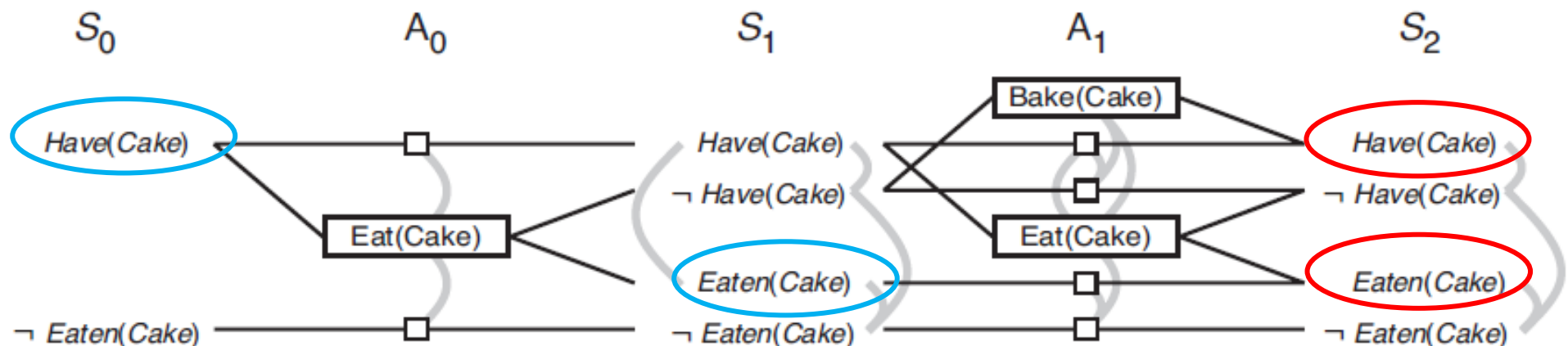
A 利用规划图进行启发式估计

单个目标的估计代价

- 从状态 s 获得任何目标文字 g 的代价为在从初始状态 s 开始构建出的规划图中 g 首次出现的层, 称为 g 的**层次代价** (level cost) 。
- max?
- sum?

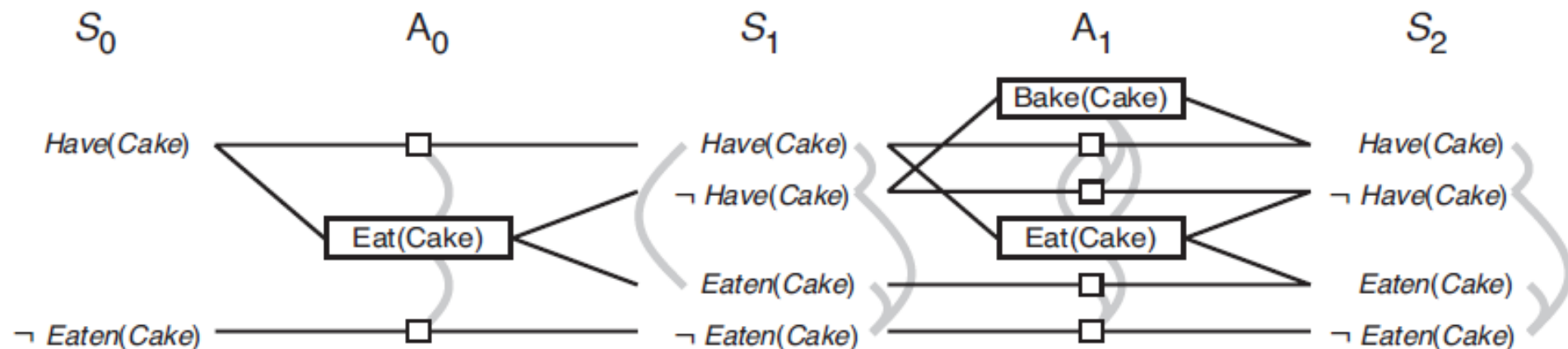
合取目标的估计代价

- **最大层** (max-level) 启发式简单取任何目标的最大层次代价；这个启发式是可采纳的。
- **层次和** (level sum) 启发式返回目标的层次代价之和；这个启发式可能不是可采纳的，但在实际中对于可大规模分解的问题效果很好。



合取目标的估计代价

- **集合层次** (set-level) 启发式找到**合取目标**中的所有文字出现在规划图中的层次，这一层没有任何一对目标文字是互斥的。





B 从规划图提取规划: GRAPHPLAN

从规划图提取规划: GRAPHPLAN

- GRAPHPLAN算法反复地用EXPAND-GRAPH向规划图增加一层。一旦所有目标在图中出现，没有互斥，GRAPHPLAN就调用EXTRACT-SOLUTION搜索解决问题的规划。

function GRAPHPLAN(problem) **returns** solution or failure

graph ← INITIAL-PLANNING-GRAPH(problem) 初始化规划图

goals ← CONJUNCTS(problem.GOAL)

nogoods ← an empty hash table

for tl = 0 **to** ∞ **do**

if goals all non-mutex in S_t of graph **then** 如果所有目标出现，没有互斥

 solution ← EXTRACT-SOLUTION(graph, goals,
 NUMLEVELS(graph), nogoods)

if solution $\neq$ failure **then return** solution

if graph and nogoods have both leveled off **then return** failure

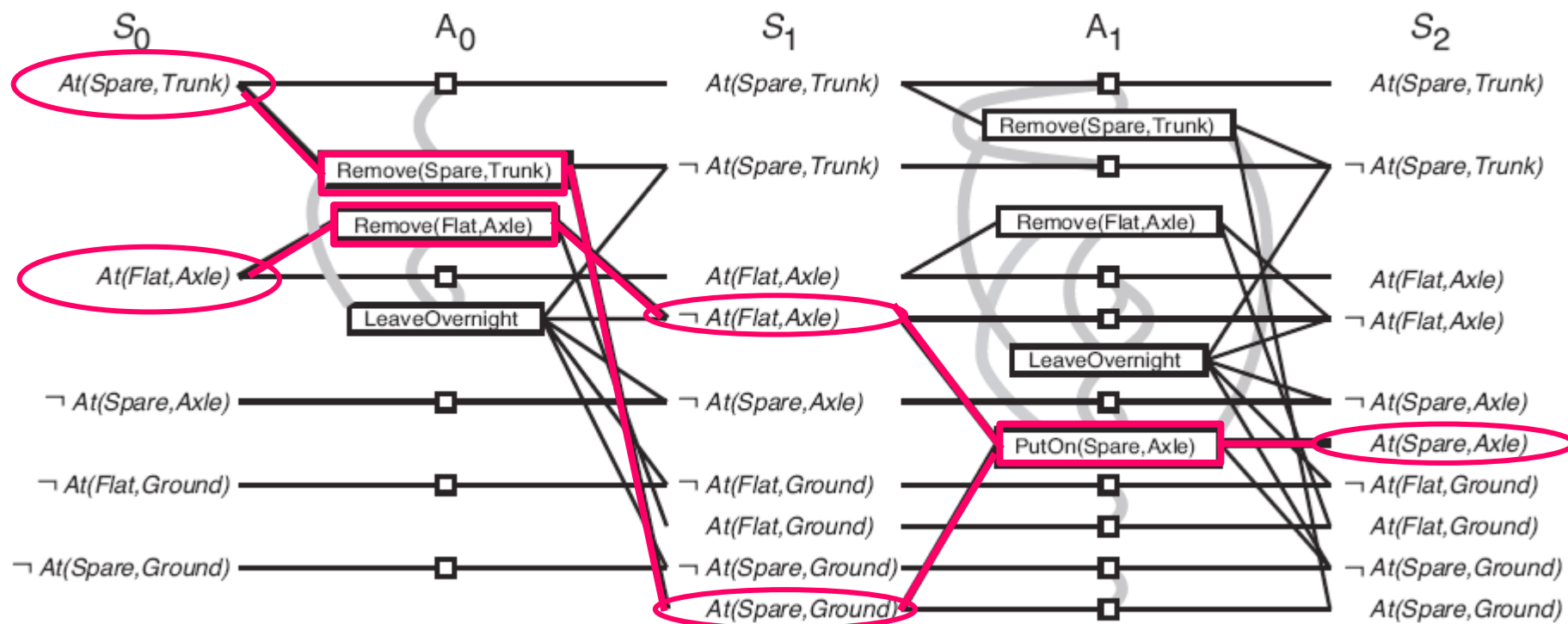
 graph ← EXPAND-GRAPH(graph, problem) 扩展规划图

备用轮胎问题

Init(Tire(Flat) $\wedge$ Tire(Spare) $\wedge$ At(Flat, Axle) $\wedge$ At(Spare, Trunk))
Goal (At(Spare, Axle))
Action(Remove(obj, loc),
 PRECOND: At(obj, loc)
 EFFECT: $\neg$ At(obj, loc) $\wedge$ At(obj, Ground))
Action(PutOn(t, Axle),
 PRECOND: Tire(t) $\wedge$ At(t, Ground) $\wedge$ $\neg$ At(Flat, Axle)
 EFFECT: $\neg$ At(t, Ground) $\wedge$ At(t, Axle))
Action(LeaveOvernight,
 PRECOND:
 EFFECT: $\neg$ At(Spare, Ground) $\wedge$ $\neg$ At(Spare, Axle)
 $\wedge$ $\neg$ At(Spare, Trunk) $\wedge$ $\neg$ At(Flat, Ground)
 $\wedge$ $\neg$ At(Flat, Axle) $\wedge$ $\neg$ At(Flat, Trunk))

反复用EXPAND-GRAPH扩展出规划图

$\text{Init}(\text{Tire}(\text{Flat}) \wedge \text{Tire}(\text{Spare}) \wedge \text{At}(\text{Flat}, \text{Axle}) \wedge \text{At}(\text{Spare}, \text{Trunk}))$
 $\text{Goal}(\text{At}(\text{Spare}, \text{Axle}))$
 $\text{Action}(\text{Remove}(\text{obj}, \text{loc}),$
 $\text{PRECOND: At}(\text{obj}, \text{loc})$
 $\text{EFFECT: } \neg \text{At}(\text{obj}, \text{loc}) \wedge \text{At}(\text{obj}, \text{Ground}))$
 $\text{Action}(\text{PutOn}(\text{t}, \text{Axle}),$
 $\text{PRECOND: Tire}(\text{t}) \wedge \text{At}(\text{t}, \text{Ground}) \wedge \neg \text{At}(\text{Flat}, \text{Axle})$
 $\text{EFFECT: } \neg \text{At}(\text{t}, \text{Ground}) \wedge \text{At}(\text{t}, \text{Axle}))$
 $\text{Action}(\text{LeaveOvernight},$
 PRECOND:
 $\text{EFFECT: } \neg \text{At}(\text{Spare}, \text{Ground}) \wedge \neg \text{At}(\text{Spare}, \text{Axle}) \wedge \neg \text{At}(\text{Spare}, \text{Trunk}) \wedge \neg \text{At}(\text{Flat}, \text{Ground}) \wedge \neg \text{At}(\text{Flat}, \text{Axle}) \wedge \neg \text{At}(\text{Flat}, \text{Trunk}))$





EXTRACT-SOLUTION如何找到解?

```
function GRAPHPLAN(problem) returns solution or failure
  graph  $\leftarrow$  INITIAL-PLANNING-GRAPH(problem)
  goals  $\leftarrow$  CONJUNCTS(problem.GOAL)
  nogoods  $\leftarrow$  an empty hash table
  for tl = 0 to  $\infty$  do
    if goals all non-mutex in  $S_t$  of graph then
      solution  $\leftarrow$  EXTRACT-SOLUTION(graph, goals,
                                         NUMLEVELS(graph), nogoods)
    if solution  $\neq$  failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph  $\leftarrow$  EXPAND-GRAPH(graph, problem)
```

如果所有目标出现，没有互斥

Extract-Solution退出时nogoods里保存着哪些目标没有实现的标注

EXTRACT-SOLUTION

构想为一个约束满足问题

- 扩展出备用轮胎问题的规划图后，所有目标文字都在 S_2 中，它们都不互斥，意味着一个解可能出现了，EXTRACT-SOLUTION将试图找到这个解。
- 可以将EXTRACT-SOLUTION构想为一个布尔约束满足问题（CSP），其中变量是在每一层的动作，每个变量的值是“在规划之内”或“在规划之外”，约束是互斥以及需要满足每个目标和前提。

变量、值域、约束

EXTRACT-SOLUTION

定义为一个后向搜索问题

- 或者，将EXTRACT-SOLUTION定义为一个后向搜索问题，搜索中的每个状态含有指向规划图某一层的指针、以及未被满足的目标集合。
- 这个后向搜索的初始状态、给定状态下的可用动作、转移模型、目标状态是什么？

初始状态,可用动作,转移模型,目标测试,动作代价

EXTRACT-SOLUTION

定义为一个后向搜索问题

定义这个后向搜索问题如下：

- **初始状态**是最后一层 S_n ，以及来自规划问题的目标集合。
- 在 S_i 层的状态中**可用动作**要选择 A_{i+1} 中的某个无冲突的、效果覆盖 S_i 层目标集合的动作子集。
- **结果状态**的层为 S_{i+1} ，并将选择的动作集合的前提作为目标集合。“无冲突”是指一组动作任意两个都不是互斥的，它们的任意两个前提也不是互斥的。
- **目标**是达到一个在 S_0 的状态，使所有目标得到满足
- 每个动作的**代价**是1。

初始状态,可用动作,转移模型,目标测试,动作代价



C. GRAPHPLAN的终止



GRPAHPLAN何时终止?

```
function GRAPHPLAN(problem) returns solution or failure
  graph  $\leftarrow$  INITIAL-PLANNING-GRAPH(problem)
  goals  $\leftarrow$  CONJUNCTS(problem.GOAL)
  nogoods  $\leftarrow$  an empty hash table
  for  $tl = 0$  to  $\infty$  do
    if goals all non-mutex in  $S_t$  of graph then
      solution  $\leftarrow$  EXTRACT-SOLUTION(graph, goals,
                                         NUMLEVELS(graph), nogoods)
    if solution  $\neq$  failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph  $\leftarrow$  EXPAND-GRAPH(graph, problem)
```

Extract-Solution退出时nogoods里保存着哪些目标没有实现的标注

要扩展多远？

- 规划图达到稳定后，还要继续扩展多远？
- 如果函数EXTRACT-SOLUTION没能找到解，那么至少有一组目标没有实现并被标注为no-good。如果下一层里no-good的数量可能更少，那么我们应该继续。一旦图和no-good都达到稳定，而又没有找到解，我们就可以失败而终止

THANKS
